

POSTECH — FIAP

Pós-Graduação em Tecnologia

Tech Challenge — Fase 4

Relatório de Entrega

Observabilidade Total e Resposta Ativa

OpenTelemetry · Prometheus · Loki · Grafana · Datadog · PagerDuty · Self-Healing

Philipe de Oliveira Marchezini

RM: 369453

Ambiente: AWS Academy (Opção A)

1. Identificação

Aluno	Philippe de Oliveira Marchezini
RM	369453
Discord	philipemarchezini
Repositório	github.com/PhilippeMarchezini/entrega_3
Vídeo	(link será inserido)

2. Resumo da Entrega

Esta fase adiciona observabilidade completa sobre a arquitetura construída nas fases anteriores: cinco microsserviços containerizados, infraestrutura provisionada por Terraform, pipelines DevSecOps no GitHub Actions e deploy no EKS gerenciado por GitOps com ArgoCD.

O problema descrito no enunciado — falha silenciosa no evaluation-service, seis horas até a equipe ser avisada e mais quatro procurando a causa em logs espalhados — foi atacado em quatro frentes: telemetria padronizada com OpenTelemetry, métricas e logs centralizados em Prometheus, Loki e Grafana, visibilidade profunda com APM comercial, e resposta automática a incidentes com alerta, plantão, ChatOps e self-healing.

3. Arquitetura do OpenTelemetry

O OTEL Collector é a peça central da telemetria. As aplicações não conhecem os backends: emitem OTLP para um único endereço, e o Collector roteia cada sinal para o destino correto.

```
aplicações (OTel SDK) --OTLP--> otel-collector (Deployment)
  traces -> Datadog (APM, Service Map)
  metrics -> Datadog + exporter Prometheus (:8889)
```

```
logs dos pods --> otel-collector-logs (DaemonSet, filelog)
  logs -> Loki (OTLP nativo) + Datadog
```

```
Grafana <- datasources: Prometheus e Loki
```

São dois Collectors por uma razão prática: log em arquivo é local ao nó e exige DaemonSet, enquanto o OTLP das aplicações precisa de um endereço estável, o que pede um Deployment com Service.

A instrumentação foi feita de duas formas. Nos serviços em Go (auth e evaluation), manualmente, com o SDK do OpenTelemetry e o handler otelhttp — Go é compilado e não permite injeção em tempo de execução. Nos três serviços em Python, de forma automática, com o opentelemetry-instrument no Dockerfile, sem alterar uma linha do código da aplicação.

No evaluation-service foi necessário propagar o context.Context da requisição por toda a cadeia de

chamadas até as requisições HTTP de saída. Sem isso, cada serviço apareceria como um trace isolado e o Service Map não se formaria.

Benefício da abordagem: trocar o APM significa alterar um exporter no Collector, versionado no Git. Nenhuma aplicação é recompilada, e não há lock-in no SDK proprietário de um fornecedor.

4. Monitoramento Opensource (Métricas e Logs)

Toda a stack foi adicionada ao repositório GitOps como Applications do ArgoCD que instalam Helm charts — o ArgoCD passa a gerenciar o próprio monitoramento.

- **Prometheus:** via kube-prometheus-stack, com node-exporter e kube-state-metrics. Faz scrape do exporter Prometheus do OTel Collector.
- **Loki:** modo SingleBinary com storage em filesystem. O modo distribuído exigiria bucket S3 e, com ele, uma IAM Role — impossível no AWS Academy.
- **Grafana:** datasources, dashboard, regra de alerta e contact points provisionados como código.

O dashboard customizado reúne recursos do cluster (CPU e memória por nó, pods por deployment), a taxa de requisições e a latência p95 por microsserviço, a taxa de erros 5xx do auth-service e um painel de logs em tempo real vindo do Loki.

5. Alertas Inteligentes e Self-Healing

O alerta dispara quando a taxa de respostas 5xx do auth-service ultrapassa 5% numa janela de cinco minutos, sustentada por pelo menos um minuto. A espera do 'for' é deliberada: alerta que dispara em pico momentâneo vira alerta que ninguém lê — combate direto a alert fatigue.

Um detalhe encontrado durante a implementação merece registro. O endpoint /validate do auth-service convertia qualquer erro de consulta — inclusive banco de dados inacessível — em HTTP 401. O serviço respondia 'chave inválida' quando o problema real era de infraestrutura, e nenhum 5xx era emitido. Era exatamente a falha silenciosa descrita no enunciado. O código foi corrigido para distinguir chave inexistente (401) de falha de infraestrutura (503), o que tornou o problema observável e o alerta possível.

Quando o alerta entra em Firing, três ações acontecem em paralelo:

1. **PagerDuty:** incidente aberto automaticamente via Events API v2, com severidade crítica.
2. **Discord (ChatOps):** notificação detalhada no canal do time, com alerta, serviço e runbook acionado.
3. **Self-Healing:** webhook para a API do GitHub (repository_dispatch) aciona um workflow que autentica na AWS, obtém o kubeconfig do EKS e executa kubectl rollout restart no deployment afetado, registrando o estado dos pods antes e depois.

Limite consciente da automação: o restart mitiga a classe de falha que restart resolve — memory leak, conexão presa, estado corrompido em memória. Quando a causa raiz é externa, como um firewall fechado, o restart não resolve, o incidente permanece aberto no PagerDuty e o plantonista assume. A automação não substitui o on-call: ela compra tempo para ele.

6. Evidências Visuais

Evidência 1 — Dashboard do Grafana

[PRINT AUSENTE: evidencias/01-grafana-dashboard.png]

Dashboard customizado 'ToggleMaster - Visão Geral', versionado no repositório como ConfigMap e importado automaticamente pelo sidecar do Grafana. Exibe recursos do cluster, taxa de requisições e latência p95 por microsserviço (métricas vindas do OpenTelemetry) e painel de logs em tempo real via Loki.

Evidência 2 — Trace distribuído no APM (Datadog)

[PRINT AUSENTE: evidencias/02-datadog-trace.png]

Trace de uma requisição em /evaluate. A cascata mostra o span de entrada no evaluation-service e as chamadas paralelas ao flag-service e ao targeting-service, comprovando a propagação de contexto entre serviços.

Evidência 3 — Notificação de incidente no ChatOps (Discord)

[PRINT AUSENTE: evidencias/03-discord-chatops.png]

Notificação detalhada enviada ao canal do Discord pelo contact point do Grafana no momento em que o alerta entrou em Firing, informando o alerta, o serviço afetado e a ação automática acionada.

Evidência 4 — Execução da automação de Self-Healing

[PRINT AUSENTE: evidencias/04-self-healing.png]

Log do workflow 'Self-Healing' no GitHub Actions, disparado por repository_dispatch a partir do webhook do Grafana. Executa o rollout restart do deployment afetado e registra o estado dos pods antes e depois da ação corretiva.

7. Justificativa das Escolhas

7.1 Datadog vs New Relic

Ambos recebem OTLP nativamente, o que dispensa agente proprietário no cluster — vantagem relevante no AWS Academy, onde não é possível criar roles de IAM. O Datadog foi escolhido pela riqueza do Service Map e por ser a ferramenta mais difundida no mercado brasileiro, o que dá valor prático ao aprendizado.

A desvantagem é o trial de catorze dias, enquanto o New Relic oferece free tier permanente de 100 GB por mês. Como quem conversa com o APM é o Collector, e não a aplicação, migrar significa trocar um exporter e reaplicar o manifesto.

7.2 PagerDuty vs Opsgenie

O Opsgenie foi absorvido pelo ecossistema Atlassian e o cadastro gratuito independente foi descontinuado, o que inviabiliza o uso em um projeto acadêmico. O PagerDuty mantém plano gratuito para times pequenos e sua Events API v2 é aceita nativamente pelo contact point do Grafana, sem intermediários.

7.3 Discord como ChatOps

O Grafana tem contact point nativo para Discord, e o webhook de canal é criado em segundos, sem depender de aprovação administrativa de workspace — diferente do Slack, que exige criar e instalar um app.

7.4 GitHub Actions como motor de Self-Healing

A automação reaproveita os secrets de AWS já usados pelos cinco pipelines e produz um log de execução auditável na aba Actions — evidência clara de que a ação corretiva ocorreu. As alternativas avaliadas (AWS Lambda e operador no cluster) exigiriam permissões de IAM indisponíveis no Academy ou expor um endpoint adicional.

8. Desafios Encontrados

- **Falha silenciosa no auth-service:** erro de banco mascarado como 401. Sem a correção, o alerta de 5xx nunca dispararia.
- **Propagação de contexto em Go:** foi preciso alterar a assinatura de toda a cadeia de funções do evaluation-service para que o trace atravessasse os serviços.
- **Restrições do AWS Academy:** sem IAM Roles, o Loki roda com storage em disco em vez de S3, e o roteamento ao APM depende de API key em Secret em vez de IRSA.
- **Capacidade do cluster:** a stack de observabilidade não coube no node group original de 2x t3.medium; foi necessário redimensionar para 3x t3.large.
- **Payload do webhook de Self-Healing:** o repository_dispatch do GitHub exige o campo event_type, incompatível com o corpo padrão do webhook do Grafana. Resolvido com custom payload; um relay em cluster foi versionado como alternativa.

9. Código no Repositório

- gitops/monitoring/ — Applications do ArgoCD para kube-prometheus-stack, Loki e os dois OTel Collectors, além do dashboard customizado
- auth-service-main/otel.go e evaluation-service-main/otel.go — instrumentação manual dos serviços em Go
- */Dockerfile e */requirements.txt — instrumentação automática dos serviços em Python
- .github/workflows/self-healing.yml — automação de resposta ao incidente
- gitops/monitoring/setup-secrets.sh — criação dos Secrets fora do versionamento
- docs/roteiro-video-fase4.md — roteiro da demonstração
- terraform/ — infraestrutura como código das fases anteriores, atualizada

Nenhuma credencial foi versionada: chaves de API, integration key do PagerDuty, webhook do Discord e token do GitHub são injetados por Secrets do Kubernetes criados fora do Git.